

Building the AArch32 and AArch64 Ports

From clone to twenty-four images, QEMU and hardware

Toolchains: arm-none-eabi-gcc 15, aarch64-none-elf-gcc 15

Boards: Raspberry Pi Zero 2 W, Raspberry Pi 3 B (BCM2837)

Emulator: QEMU raspi3b, 4 cores

Suites: 11 passed, 1 skipped, 0 failed on both

Building the AArch32 and AArch64 ports

Everything needed to go from empty directory to twenty-four test images per architecture, running under QEMU or on a Raspberry Pi.

1. Get the sources

Each architecture project keeps **no copy** of the kernel or the drivers. Clone what you need side by side:

```
mkdir workspace && cd workspace
git clone <remote>/micro-os-plus-iii-smp.git
git clone <remote>/micro-os-plus-iii-devices.git
git clone <remote>/micro-os-plus-iii-aarch64.git      # and/or -aarch32
```

```
workspace/
├─ micro-os-plus-iii-smp/
├─ micro-os-plus-iii-devices/
├─ micro-os-plus-iii-aarch64/
└─ micro-os-plus-iii-aarch32/
```

CMake finds them as siblings. Override either with

```
cmake -DUOS_SMP_DIR=/elsewhere/micro-os-plus-iii-smp \
      -DUOS_DEVICES_DIR=/elsewhere/micro-os-plus-iii-devices ...
```

A build that cannot find one stops and names the repository to clone.

One kernel working copy serves every architecture project on the machine, so an edit to it is visible to all of them at once — no submodule pointer to bump, no commit–push–pull round trip.

2. Tools

Tool	Used for	Notes
<code>arm-none-eabi-gcc</code> 15	aarch32	xPack
<code>aarch64-none-elf-gcc</code> 15	aarch64, and the AArch32 QEMU boot shim	xPack
CMake $\geq$ 3.20	both	
<code>qemu-system-aarch64</code>	both suites	xPack QEMU 9.x
python3 + <code>markdown</code> , <code>weasyprint</code>	<code>docs/md2pdf.py</code>	docs only

Toolchains are found under `~/.local/xPacks/@xpack-dev-tools/`, newest first, falling back to `PATH`. Pin one exactly with

```
cmake -DUOS_TOOLCHAIN_BIN=~/.local/xPacks/@xpack-dev-tools/aarch64-none-elf-gcc/
15.2.1-1.1.1/.content/bin ...
```

3. Configure and build

AArch64 — Raspberry Pi Zero 2 W / Pi 3 B, 4× Cortex-A53, ARMv8-A

```
cd micro-os-plus-iii-aarch64
cmake -S . -B build \
    -DCMAKE_TOOLCHAIN_FILE=../micro-os-plus-iii-smp/cmake/toolchains/aarch64-none-elf.cmake
cmake --build build -j8
```

AArch32 — same silicon in 32-bit mode, ARMv7-A

```
cd micro-os-plus-iii-aarch32
cmake -S . -B build \
    -DCMAKE_TOOLCHAIN_FILE=../micro-os-plus-iii-smp/cmake/toolchains/arm-none-eabi.cmake
cmake --build build -j8
```

Board

```
cmake -S . -B build-rpi3b -DBOARD=rpi3b ...      # default: zero2w
```

`rpi3b` selects `linker-rpi3b.ld` and defines `BOARD_RPI3B`.

What you get

24 targets — twelve applications × {qemu, hwd} — each producing an ELF, a .bin (the kernel8.img / kernel7.img equivalent) and a .map, all in build/test/.

```
sd_test-qemu      sd_test-hwd
smp_test0..4-qemu  smp_test0..4-hwd
smp-mat-test-qemu  smp-mat-test-hwd
smp-mat-sdcard-test-... smp-num-test-...
smp-pipeline-test-... smp-pro-cons-test-...
usb_test-qemu      usb_test-hwd
```

qemu is the emulator build. hwd adds HW_BUILD (SD tests use the existing FAT32 boot partition rather than formatting a blank card) and DEBUG_BOOT (early-boot asm markers for OpenOCD bring-up).

4. Running the QEMU suites

One runner serves every architecture; the caller supplies the machine.

AArch64 — straight to -kernel

```
../micro-os-plus-iii-smp/test/run-qemu.sh build/test \
    "${ls ~/.local/xPacks/@xpack-dev-tools/qemu-arm/*.content/bin/qemu-system-aarch64 | tail -1}" \
    -M raspi3b -smp 4
```

AArch32 — through the boot shim

QEMU's raspi3b starts its Cortex-A53 cores in AArch64, so a 20-line stub drops to AArch32 and jumps to the image. CMake builds it as qemu-shim.

```
UOS_QEMU_SHIM=build/test/shim8.img UOS_QEMU_LOAD_ADDR=0x10000 \
../micro-os-plus-iii-smp/test/run-qemu.sh build/test \
    "${ls ~/.local/xPacks/@xpack-dev-tools/qemu-arm/*.content/bin/qemu-system-aarch64 | tail -1}" \
    -M raspi3b -smp 4
```

Not raspi2b. The port is built -mcpu=cortex-a53, so its load-acquire instructions are undefined on the raspi2b Cortex-A7 and the image faults at the first one. Hardware needs no shim at all: config.txt sets arm_64bit=0 and the cores come up in AArch32 directly.

The runner creates the SD images the four card tests need — a seeded flatfs volume for sd_test, a blank 4 GiB image for the others — under build/test/.qemu-logs/.

Run one suite at a time. Two four-core suites at once, or one alongside a build, starves a QEMU vCPU: a worker freezes mid-loop with no fault, which looks exactly like a deadlock. Re-run on an idle host before investigating.

Result

```
qemu suite: 11 passed, 1 skipped, 0 failed
```

on both architectures. `usb_test` skips by design — QEMU emulates no USB device mode — exactly as the predecessor suite recorded it.

5. Running on hardware

Flash a `*-hwd.bin` as `kernel8.img` (AArch64) or `kernel7.img` (AArch32) on the boot partition, with the port's `config.txt` from `boards/rpi-zero-2w/`. For a debug-in-RAM run, the OpenOCD configurations for J-Link and Olimex are in the same directory.

6. Declaring your own application

```
uos_add_app (my_app
  SOURCES      main.cpp
  PORT         micro-os-plus::aarch64
  LIBRARIES     micro-os-plus::devices
  LINKER_SCRIPT ${CMAKE_CURRENT_SOURCE_DIR}/linker.ld
  NCPU         4
  DEFINES      TRACE __ARM_EABI__ SEMIHOST
)
```

`uos_add_app` links the kernel, sets `OS_NCPU` (and `OS_USE_SMP_SCHEDULER` when it exceeds 1), applies the bare-metal flag set, adds the linker script, and emits the `.bin` and a size listing. It replaces the 214 Makefiles the predecessor repository carried, which differed by about 48 lines of boilerplate each.

To build one of the shared tests instead:

```
uos_add_test_app (smp_test1 NCPU 4 LINKER_SCRIPT ... DEFINES ...)
```

which resolves the sources from `test/common/smp_test1/`. No build ever spells out a path into the test tree.

7. CMake targets exported

micro-os-plus-iii-smp

Target	Contents
micro-os-plus::iii	the kernel core: 37 sources every port compiles
micro-os-plus::iii-posix-io	13 sources, the file-descriptor layer
micro-os-plus::iii-drivers	2
micro-os-plus::iii-startup	the generic reset path — a port with its own <code>startup.S</code> skips it
micro-os-plus::iii-newlib-reent	1
micro-os-plus::iii-semihosting	1
micro-os-plus::iii-trace-itm / -trace-semihosting / -trace-segger-rtt	mutually exclusive; link at most one
micro-os-plus::port-smp-decls	the shared <code>os-decls.h</code>
micro-os-plus::test-common	the shared test support

The ARM ports link the core, `port-smp-decls` and `test-common`, and **none** of the optional groups. Linking `iii-posix-io` into a newlib bare-metal build fails to compile: it declares `read` / `write` returning `ssize_t` where newlib declares `int`.

micro-os-plus-iii-devices

Target	Contents
micro-os-plus::devices	SD, flatfs, DWC2, FatFs
micro-os-plus::soc-bcm2837	BCM2837 registers, IRQ, mailbox

The architecture projects

`micro-os-plus::aarch32`, `micro-os-plus::aarch64` — each links `micro-os-plus::iii`, `micro-os-plus::port-smp-decls` and `micro-os-plus::soc-bcm2837`, and carries its `-mcpu` as both a compile and a **link** option. Without the machine flags at link time the driver picks the wrong multilib and every AArch32 link fails with *"uses VFP register arguments"*.

8. Defines

See `smp-construction.md` §5 for the complete table, per folder: what the port header fixes, what `uos_add_app` sets, what each architecture project's `test/CMakeLists.txt` adds, and which of the kernel's many optional `OS_*` switches matter (none of them, for these builds).

9. Rebuilding this document

```
python3 docs/md2pdf.py docs/building-aarch32-aarch64.md docs/building-aarch32-aarch64.pdf \
  --title "Building the AArch32 and AArch64 ports" \
  --subtitle "μOS++ III SMP" \
  --footer "μOS++ III SMP – build guide"
```